

Pipeline de recommandation

Pick Your Spotlight — Documentation technique

Moteur cœur · Surprise du soir · Juin 2026

Version document : 25 juin 2026

Sources : RECOMMENDATION_PIPELINE.md · BACKLOG.md

Table des matières

- 1. Vue d'ensemble
- 2. Architecture end-to-end
- 3. Étapes du pipeline
- 4. Fichiers source
- 5. Filtres et cascade SQL
- 6. Chaîne de fallbacks
- 7. Paramètres et constantes
- 8. Échelles de score
- 9. Console debug [PICK-DEBUG]
- 10. État TNR et tests
- 11. Performance
- 12. Quirks connus
- 13. Prérequis et garde-fous
- 14. Autres points d'entrée
- 15. Recommandations de maintenance

1. Vue d'ensemble

Le pipeline de recommandation est le moteur cœur de Pick. Il transforme le profil de goût d'un utilisateur en 1 à 5 films affichés sur l'écran d'accueil (« Ce soir »), avec textes personnalisés et score de match.

Le pipeline « surprise du soir » s'active quand l'utilisateur a "e 2 films likés et un vecteur de goût 32D calculé. Sinon, un fallback TMDB léger (getSurpriseRecommendation) complète le batch côté client.

Entrées et sorties

Type	Éléments
Entrées	Profil (tasteProfile), vecteurs stable/recent/avoidance, plateformes, genres exclus, excludeIds, overrides voix/ambiance/duo
Sortie	3 films typiques (recommendationCount défaut 5) : affiche TMDB, reason LLM, score fusionné, textes movie-match

2. Architecture end-to-end

```
CLIENT (React)
  HomeScreen.generateTonightPick
    !' taste-engine.ts (computeMultiVectorProfile)
    !' invoke surprise-personalized (debug: true)
  recommendation-batch.ts !• movies[] + debugData
    !' onBatchReady: 3 films LLM !' UI
    !' preloadMatchTexts séquentiel !' movie-match

EDGE: surprise-personalized
  1. SQL vectoriel cascade (TARGET ~100, plateforme ON)
  1.4 SQL explicite si pool insuffisant
  1.7 Enrichissement langue TMDB
  2. Top 50 score composé
  2.2 Gemini – sélection N films
  3. Enrichissement TMDB batch
  4. discover-fallback si échec

EDGE: movie-match
  Scoring + textes riches – séquentiel par film
```

3. Étapes du pipeline

#	Étape	Comptes	Critères
0	Préparation client	excludeIds 20–200+	Feedback, rejets session, duo
1	SQL vectoriel 32D	Pool ~100	RPC match_movies_for_recommendation, cascade 0!'3
1.4	SQL explicite	Jusqu'à 100	match_movies_explicit, 3 niveaux
1.7	Enrich. langue	60 max	TMDB original_language null
2	Top composite	50 films	composite = sim×100 + note (+15 langue)

2.2	LLM Gemini	50!'count+2	JSON selections, fallback déterministe
3	TMDB enrich	1/film	getMovieDetails, retry movie/tv
3.5	Retry qualité	+N	Si < 3 films "e minMatchScore
4	discover-fallback	0-3	Trending TMDB with_watch_providers
5	Affichage UI	3 visibles	onBatchReady sans attendre MM
6	movie-match	1/film	Embedding 32D + Gemini, fusion SP/MM

4. Fichiers source

Composant	Chemin	Rôle
Profil multi-vecteurs	src/lib/taste-engine.ts	computeMultiVectorProfile, cache user_taste_vectors
Batch client	src/lib/recommendation-batch.ts	ensureRecommendationBatch, enrichissement MM
Orchestration accueil	src/components/pick/HomeScreen.tsx	generateTonightPick, [PICK-DEBUG]
Moteur edge	supabase/functions/surprise-personalized/	SQL !' LLM !' TMDB
Textes finaux	supabase/functions/movie-match/	Score 55-99 %, textes riches
TNR	src/test/recommendation-non-regression.test.ts	Invariant exclusions
Fonctions pures SP	src/lib/recommendation-pipeline.ts	23 tests unitaires

5. Filtres et cascade SQL

La RPC `match_movies_for_recommendation` applique une similarité cosinus sur embeddings 32D. Le filtre plateforme n'est jamais levé. Les niveaux 0!3 relâchent progressivement langue, année, genres, note minimale.

id debug	Étape	Description
2a-post-filter-origin	Langues exclues	Filtre langues d'origine non souhaitées
2b-post-filter-voiceGenres	Genre vocal	Post-filtre genre vocal (seuil "e 5)
2c-post-filter-decade	Décennie	voiceDecade (hard) ou profileDecades (soft)
5-final-safety	Filet final	Tri origine + filet genre/langue

6. Chaîne de fallbacks

- Client — `getSurpriseRecommendation` si < 2 likes ou pas de vecteur
- 1.4 SQL explicite — vecteur absent ou pool SQL faible (3 niveaux)
- 2.2 LLM déterministe — si Gemini KO : sélection par score composite
- 2.5 Retry qualité — complète depuis pool SQL si < 3 films "e `minMatchScore`
- 4 discover-fallback — trending TMDB with `_watch_providers`
- 4b trending-fallback — trending/popular TMDB
- 4c nuclear-fallback — genre/note levés, plateforme conservée
- movie-match FALLBACK — Gemini KO !' { `fallback: true` }, retry 4 s, films réserve

7. Paramètres et constantes

Paramètre	Valeur	Description
TARGET	100	Pool SQL cible
llmPoolSize	50	Top N envoyés au LLM
BATCH	500	Taille par round SQL
recommendationCount	5 (défaut)	Films demandés ; UI affiche typ. 3
LLM sortie	count + 2	Marge qualité (ex. 5 si count=3)
Boost langue	+15	Sur score composite pré-LLM
Lang enrich max	60	Candidats <code>original_language</code> null
debug	true (HomeScreen)	Active <code>debugData</code> et [PICK-DEBUG]

8. Échelles de score

Échelle	Où	Plage	Usage
Sim%	SQL / debug	~0–100	Tri candidats debug
Composite	Pré-LLM	sim×100 + note	Top 50 avant Gemini
LLM matchScore	Gemini	60–99	Teaser onBatchReady
movie-match	Client final	55–99	Badge · fusion max(SP"e60, MM)

Attention : les trois échelles ne sont pas directement comparables. Fusion client : ignore SP < 60 %, prend le meilleur entre SP valide et MM.

9. Console debug — [PICK-DEBUG]

Activé quand debug: true est envoyé à surprise-personalized (toujours en prod sur HomeScreen).

Groupe console	Champ debugData	Contenu
Paramètres par étape	pipelineStages	Params, comptes entrée/sortie, fallback
SQL vectoriel 32D	sqlCandidates, sqlCascadeLevel	Candidats Sim%, snippet RPC
Détail cascade	sqlLevelDebug	Films par niveau 0–3
SQL explicite	explicitFallbackDebug	Complément sans vecteur
Top N !' LLM	top50	Jusqu'à 50 films
Sélections LLM	llmSelections	matchScore + reason
Fallback	fallbackTrace	Mode discover-fallback
Timings	engineMeta.timings	SQL · lang · LLM · TMDB · fallback
Résultat MM	(client)	Score MM · rich texts vs FALLBACK

10. État TNR et tests

TNR unitaires : 131/131 OK (25 juin 2026) — dont 70 tests pipeline dédiés.

Fichier	Tests	Périmètre
taste-engine.test.ts	28	Profil multi-vecteurs
recommendation-batch.test.ts	9	Helpers batch client
recommendation-pipeline.test.ts	23	Fonctions pures SP
recommendation-batch-integration.test.ts	8	Mocks edge, fusion scores
recommendation-non-regression.test.ts	15	Invariants exclusions

Feuille de route TNR — 3 phases

Phase	État	Périmètre
1 — Unitaires	done	taste-engine, batch, pipeline
2 — Intégration	done (25/06)	Fixtures JSON, mocks edge, invariants
3 — E2E	en cours	pipeline.spec.ts, checklist SMOKE_TESTS § 3.2

Commandes : npm run test:unit (131 tests, ~10 s) · npm run test:smoke · npm run test:smoke:full

11. Performance

- Timings serveur dans engineMeta.timings : SQL, lang, LLM, TMDB, fallback
- Timings client : préfixe [Pick#ñ] pour SP et movie-match (eager/lazy)
- Affichage progressif : onBatchReady sans attendre movie-match
- movie-match séquentiel : goulot principal côté client
- Backlog 2.8 : Perf reco timeouts UX — todo

12. Quirks connus (audit juin 2026)

Quirk	Détail	Impact
reason: null	Fallback LLM, retry qualité	Raison vide en debug
Trois échelles	Sim% · LLM · MM	Ne pas trier debug par Sim%
MM FALLBACK	Gemini KO	Log FALLBACK, retry 4 s

Corrigé (1.23) : sql50/top20 !' sqlCandidates/top50 ; platformFallbackTriggered !' tasteCascadeTriggered.

13. Prérequis et garde-fous

- "e 2 likes sinon fallback getSurpriseRecommendation
- Vecteur null !' SQL explicite ou discover-fallback
- Exclusions : excludeIds + usedIds — TNR non-regression
- Plateforme toujours en SQL ; safety net client désactivé si plateformes
- Auth JWT requise (requireAuth)

14. Autres points d'entrée

Contexte	Fichier	Variante
Wizard /app	use-recommendation-engine.ts	Même SP, pas debug par défaut
Révéler soirée	HomeScreen / runRevealPipeline	Overrides mood/genres
Création événement	CreateEventPage.tsx	count: 3, pas de MM client
Duo	HomeScreen + duoUserIds	Vecteur fusionné, exclusions partenaire

15. Recommandations de maintenance

- Ne pas refactorer le pipeline sans tests 1.19 — satisfait (131 tests)
- Avant changement HomeScreen profond : npm run test:unit + non-regression
- Clôturer phase 3 TNR : pipeline.spec.ts stable, checklist SMOKE_TESTS § 3.2
- Objectif couverture 70 % src/lib/ avant refactor hooks
- Conserver cohérence noms debugData (sqlCandidates, top50, tasteCascadeTriggered)
- Perf (2.8) : timeouts UX et exploitation engineMeta.timings
- TonightPickContext (1.20) : pair humain, PR par étapes
- Après modif edge : deploy + smoke manuel § 3.2

Document généré le 25 juin 2026 · Références : RECOMMENDATION_PIPELINE.md · BACKLOG.md · SMOKE_TESTS.md

